



UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL TUCUMÁN

Trabajo Final Integrador

21/06/2026

Materia: Virtualización: Consolidación de Servidores

Comisión: 5K3

Profesores:

- Ing. Carriles, Luis Maria

Autor

- Hamada, Santino - 56070

Índice

1. Introducción y Diagnóstico Inicial del Entorno	2
2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE	2
2.1. Especificaciones Técnicas de Recursos	2
2.2. Procedimiento Paso a Paso en la Interfaz de Proxmox VE	3
3. Fase 1: Compilación Standalone y Patrón Build Server (Windows)	3
3.1. Configuración de Next.js y Vercel (CDN)	3
3.2. Proceso de Empaquetado Local	4
4. Fase 2: Transferencia de Archivos (Salto de Red Externo)	4
5. Fase 3: El Pivote Arquitectónico (Proxy Híbrido y Microservicios)	4
6. Fase 4: Despliegue en Segundo Plano (LXC 143)	5
7. Fase 5: Mitigación y Optimización de PostgreSQL (LXC 144)	5
7.1. Resolución del Conflicto de Codificación (Error 500)	5
7.2. Ajuste del Kernel y Cortafuegos de Red	5
8. Conclusión del Estado de la Arquitectura	6
9. Glosario Técnico de Comandos de Terminal	7

1. Introducción y Diagnóstico Inicial del Entorno

El presente documento expone los criterios de diseño, contingencias técnicas y el procedimiento paso a paso ejecutado para el despliegue de una solución web moderna. Inicialmente concebida como monolítica (Frontend y Backend en Next.js), la arquitectura debió pivotar hacia un modelo desacoplado de microservicios con soporte Edge (CDN). El entorno de destino corresponde a la infraestructura de la NAP (Núcleo de Asistencia de Redes) de la facultad.

El despliegue representó un desafío crítico de administración de sistemas e ingeniería de redes debido a tres condicionantes perimetrales estrictos:

- **Restricción Extrema de Hardware:** Un límite estricto e infranqueable de **128 MB de memoria RAM** utilizable por cada nodo de cómputo virtualizado.
- **Aislamiento de Red Perimetral:** Inexistencia de una red privada virtual (VPN) externa configurada en la máquina de desarrollo local, impidiendo el direccionamiento directo hacia el segmento privado universitario (172.16.90.X).
- **Políticas Estrictas del Reverse Proxy Institucional:** El proxy de entrada a la red (Nginx) presentaba bloqueos por extensión, interceptando archivos estáticos nativos de los frameworks modernos (.css, .js, .woff2) y retornando de forma sistemática el código de error HTTP 501 (*Not Implemented*).

2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE

Para garantizar el aislamiento de procesos y optimizar el uso de los recursos del hipervisor, se optó por el aprovisionamiento de dos contenedores Linux (*LXC*) independientes corriendo sobre Debian 12.

2.1. Especificaciones Técnicas de Recursos

La distribución de hardware y direccionamiento de red se configuró según los parámetros consolidados en la Tabla 1.

Cuadro 1: Aprovisionamiento de Nodos LXC en Proxmox

Componente	Servidor Backend y Proxy (LXC 143)	Servidor Base de Datos (LXC 144)
Plantilla de OS	debian-12-standard_amd64.tar.zst	debian-12-standard_amd64.tar.zst
Cómputo (vCPU)	1 Core (Limitado a host)	1 Core (Limitado a host)
Memoria RAM	128 MB	128 MB
Memoria Swap	0 MB	0 MB
Almacenamiento	8 GB (Local-SSD ext4)	8 GB (Local-SSD ext4)
Interfaz de Red	eth0 (Bridge: vmbr0)	eth0 (Bridge: vmbr0)
Dirección IP IPv4	Static: 172.16.90.143/24	Static: 172.16.90.144/24
Puerta de Enlace	172.16.90.1	172.16.90.1
DNS Server	172.16.90.1	172.16.90.1

2.2. Procedimiento Paso a Paso en la Interfaz de Proxmox VE

El aprovisionamiento de los entornos se realizó siguiendo de forma estricta la siguiente secuencia técnica dentro del panel de administración web:

1. Se navegó hasta el almacenamiento local del nodo y se verificó la existencia de la plantilla oficial de Debian 12.
2. Se hizo clic en **Create CT** (Crear Contenedor) en la esquina superior derecha del panel.
3. **Pestaña General:** Se asignó el ID del contenedor (143), el nombre de host (45275660A) y se configuró una contraseña segura para el usuario administrador **root**.
4. **Pestaña Template:** Se seleccionó la plantilla de Debian del paso 1.
5. **Pestaña Root Disk:** Se fijó el tamaño del disco en 8 GB.
6. **Pestaña CPU:** Se asignó 1 Core.
7. **Pestaña Memory:** Se configuró la memoria RAM estrictamente en 128 MB y la Swap en 0 MB.
8. **Pestaña Network:** Se configuró la tarjeta **eth0**. Se seleccionó el modo **Static** en la configuración de IPv4, ingresando el CIDR correspondiente (172.16.90.143/24) y estableciendo la Gateway en 172.16.90.1.
9. Se repitió exactamente el mismo procedimiento para el segundo contenedor asignándole el ID 144, el hostname 45275660DB y la dirección IP estática 172.16.90.144/24.

3. Fase 1: Compilación Standalone y Patrón Build Server (Windows)

El proceso tradicional de construcción en Next.js requiere la ejecución de `npm install` y `npm run build`, procesos que exigen demandas de memoria de entre 500 MB y 1 GB de RAM. Intentar ejecutarlos dentro del contenedor LXC provocaría que el kernel actúe con el *OOM Killer*, congelando el entorno. Para sortear esto, se adoptó el estándar empresarial de delegar la integración al entorno local, operando como *Build Server*.

3.1. Configuración de Next.js y Vercel (CDN)

Se dividió el proyecto: el empaquetado backend se procesa localmente, mientras que la capa de presentación (Frontend) se integra de manera continua vía GitHub a la red global Edge de Vercel. Se añadió la siguiente directiva en `next.config.js`:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'standalone',
  basePath: '/45275660',
  assetPrefix: 'https://tpf-virtualizacion.vercel.app/45275660',
}
export default nextConfig
```

3.2. Proceso de Empaquetado Local

Se diseñó un script que aísla únicamente los archivos binarios y dependencias estrictamente requeridas (reduciendo cientos de megabytes de dependencias a un empaquetado mínimo para los 128 MB de RAM disponibles):

```
npm run build
xcopy /E /I /Y public .next\standalone\public
xcopy /E /I /Y .next\static .next\standalone\static
cd .next\standalone
tar -czf ../../blog_npm.tar.gz .
```

4. Fase 2: Transferencia de Archivos (Salto de Red Externo)

A causa de la falta de VPN, la ejecución de `scp` directo arrojaba *Connection timed out*. Para instrumentar la transferencia desde Windows hacia el LXC, se aplicó una técnica de puenteo web asincrónico:

1. Se subió el archivo local `blog_npm.tar.gz` a la interfaz de Filebin (<https://filebin.net/>).
2. Se copió la URL de descarga directa y se importó en Proxmox mediante el comando `wget`.

5. Fase 3: El Pivote Arquitectónico (Proxy Híbrido y Microservicios)

Al intentar desplegar el código monolítico, el proxy inverso de la NAP comenzó a bloquear la descarga del código JavaScript y CSS (Error HTTP 501). Mudar todo a Vercel era imposible debido a que la API requería acceso a la Base de Datos privada del segmento 172.16.90.X.

La solución arquitectónica definitiva consistió en implementar un **Proxy Híbrido en Node.js** (`server-proxy.js`) en el puerto principal del contenedor (Puerto 3000) operando como enrutador inteligente:

1. **Capa de Presentación (Frontend sobre CDN):** Si el tráfico entrante solicita HTML o rutas visuales, el proxy intercepta la consulta, contacta a la CDN de Vercel en la nube y le transfiere el código directamente al navegador del cliente. Esto soluciona los conflictos de Hashes de Webpack y evade el filtro estático del Nginx de la universidad.
2. **Capa de Lógica (Headless Backend):** Si el proxy detecta que la URL pertenece a la API REST (ej. `/api/posts`), desvía internamente el tráfico al servidor puro de Next.js. El mismo, oculto en el **puerto 3001**, posee conexión limpia y segura con la base de datos PostgreSQL mediante IPs privadas.

6. Fase 4: Despliegue en Segundo Plano (LXC 143)

Para efectivizar la nueva arquitectura distribuida, se preparó el entorno inyectando el motor de ejecución y orquestando ambos procesos:

```
# 1. Instalación de Motor LTS de Node
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt install -y nodejs

# 2. Descompresión del empaquetado optimizado
wget --no-check-certificate "URL_DE_FILEBIN" -O /tmp/blog_npm.tar.gz
tar -xzf /tmp/blog_npm.tar.gz -C /opt/blog

# 3. Archivo de Entorno (.env)
DB_HOST=172.16.90.144
DB_PORT=5432
DB_NAME=blogdb
DB_USER=postgres
DB_PASSWORD=contrasena_fuerte
PORT=3001
NEXT_PUBLIC_BASE_PATH=/45275660

# 4. Lanzamiento de Microservicios Desacoplados (Nohup)
nohup env NODE_ENV=production node --env-file=.env server.js > backend.log 2>&1 &
nohup node server-proxy.js > proxy.log 2>&1 &
```

7. Fase 5: Mitigación y Optimización de PostgreSQL (LXC 144)

La plantilla del LXC 144 presentaba configuraciones regionales y parámetros de consumo de memoria totalmente incompatibles con el hardware de destino (128 MB) y el código de la aplicación.

7.1. Resolución del Conflicto de Codificación (Error 500)

El motor arrojaba bloqueos de codificación por el esquema SQL_ASCII. Se debió regenerar la estructura forzando la matriz UTF8:

```
CREATE DATABASE blogdb WITH ENCODING = 'UTF8' TEMPLATE = template0;
```

7.2. Ajuste del Kernel y Cortafuegos de Red

Se procedió a reducir drásticamente el consumo del motor editando el archivo `postgresql.conf` para adaptarlo al límite físico, bajando los buffers compartidos:

```
shared_buffers = 10MB
work_mem = 1MB
maintenance_work_mem = 8MB
```

Por último, para asegurar la integridad de la base de datos, se editó el `pg_hba.conf` para admitir tráfico únicamente desde la IP del microservicio Frontend:

host	blogdb	postgres	172.16.90.143/32	scram-sha-256
------	--------	----------	------------------	---------------

8. Conclusión del Estado de la Arquitectura

El despliegue ha sido completado con un grado de sofisticación técnica elevado, alcanzando un estado operativo óptimo sustentado bajo los principios de *Arquitectura Desacoplada* y mitigación de redes hostiles.

Gracias al rediseño basado en proxy y optimizaciones del clúster de datos, los consumos de memoria en producción se estabilizaron bajo métricas impecables:

- **Nodo Backend y Proxy (LXC 143):** Estabilizado en apenas **70 MB de memoria RAM**. Gracias al uso de la directiva Standalone de Next.js y delegando la carga estática a la nube de Vercel.
- **Nodo de Persistencia (LXC 144):** Estabilizado en **60 MB de memoria RAM**, consecuencia directa de la reducción de los buffers en PostgreSQL.

La escalabilidad e independencia lograda permiten modificar y compilar el diseño visual del proyecto a través de repositorios remotos (GitHub/Vercel) sin necesidad de impactar ni interrumpir los servicios alojados en el clúster de la NAP, cumpliendo todos los requisitos bajo el estricto límite de 128 MB asignado por la cátedra.

9. Glosario Técnico de Comandos de Terminal

Cuadro 2: Glosario de Comandos de Administración de Sistemas

Comando Técnico	Impacto y Acción Operativa en el Sistema
<code>apt update && apt install</code>	Sincroniza el mapa local de paquetes e instala dependencias requeridas.
<code>tar -czf / -xzf</code>	Comprime (c) o extrae (x) archivos mediante el algoritmo gzip.
<code>wget -no-check-certificate</code>	Descarga recursos ignorando alertas de validación SSL por proxy intermedio.
<code>nohup [proceso] &</code>	Ordena la ejecución de un subproceso de fondo de forma persistente, evitando que sea terminado (No Hang Up) al cerrar la sesión SSH.
<code>node -env-file=.env</code>	Levanta instancias del motor JavaScript pasándole archivos locales de entorno.
<code>systemctl restart</code>	Ordena a Systemd reiniciar un demonio y aplicar configuraciones.
<code>xcopy /E /I /Y [origen] [destino]</code>	Copia archivos y árboles de directorios completos de Windows de forma recursiva, forzando el reemplazo y omitiendo advertencias.
<code>rm -rf [ruta]</code>	Elimina un archivo o estructura de directorios de manera recursiva e incondicional, forzando la remoción del sistema.
<code>mkdir -p [ruta]</code>	Genera un directorio asegurando la creación de las carpetas padre anidadas sin arrojar fallos en caso de existir previamente.
<code>pkill -9 node</code>	Envía de forma masiva e irreversible la señal de interrupción forzada SIGKILL a todas las instancias activas de Node.js en memoria.
<code>node -env-file=.env server.js &</code>	Ejecuta el backend de Next.js inyectando variables de entorno físicas y ordenando al hilo correr en segundo plano.
<code>su - postgres</code>	Sustituye el entorno de usuario del shell actual para adoptar la identidad y los privilegios del administrador de PostgreSQL.
<code>psql</code>	Invoca la terminal interactiva de comandos de PostgreSQL para en- tablar consultas y transacciones sobre bases de datos relacionales.
<code>\c [nombre_base]</code>	Instrucción interna del prompt de <code>psql</code> para conmutar la sesión activa hacia el contexto de una base de datos específica.
<code>\q</code>	Finaliza y abandona la sesión interactiva del cliente de comandos de <code>psql</code> , restituyendo el control a la terminal de Linux.
<code>systemctl restart [servicio]</code>	Ordena al gestor de arranque Systemd reiniciar un demonio de fondo para forzar la lectura de nuevos archivos de configuración.